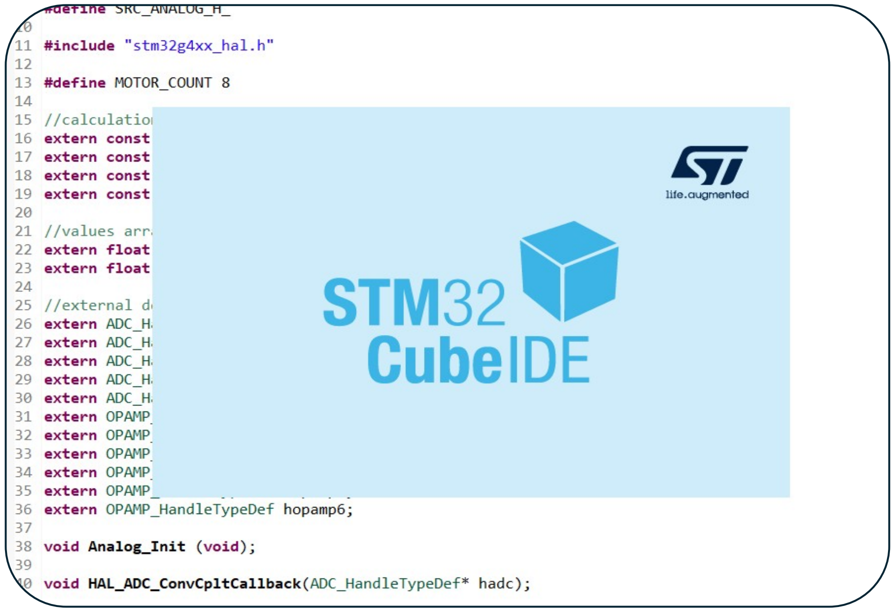
Electronic Software



Everything you need to understand the code that I produced this semester and work with it. We are working with a STm32G474 MCU either on a custom board (QET6 version) or on a nucleo (RE version)

This is C code using the libraries in Stm32cube and the code generation from CubeMX. I used the 1.19.0 version for this work.

The goal of my work was to control the different tools on the MCU and obtain reliable high level values and functions.

This code is made to work on the v2.2 board. There is a version of the Test code for nucleo too because it is a lot more convenient for a lot of tests

This pdf is a downloadable version of this notion page :
https://app.notion.com/p/n-pulse/Software-2a1e4f65ffef80c39081e30f81aa752e

Index

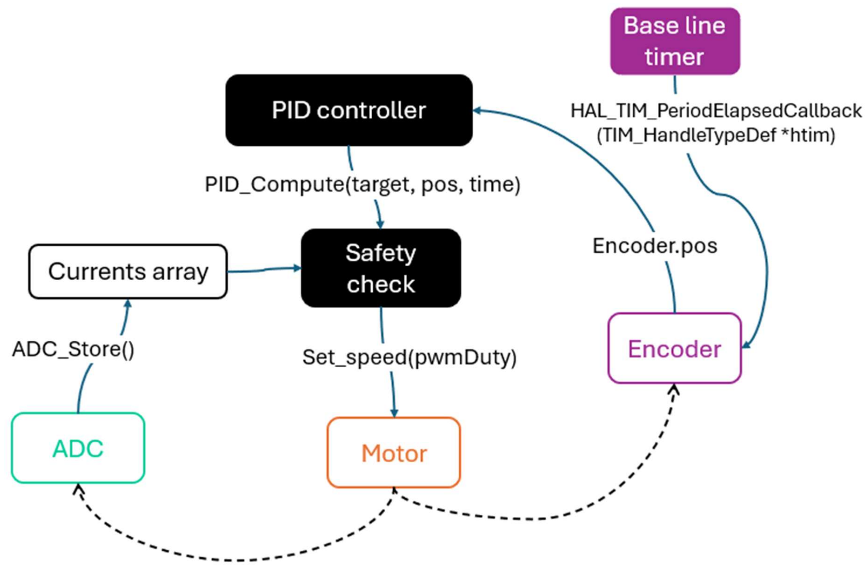
1) Architecture
    a. Diagrams 3
    b. Notes 6
2) Functions and pins
    a. Analog 7
    b. Encoders 8
    c. PWM 9
    d. Communication 10
    e. Nucleo Pinout 11
3) CubeMX Set up
    a. Debug 12
    b. ADC (analog converter) 13
    c. OPAMP (signal amplificator) 15
    d. High Resolution Timer (for PWM) 16
    e. Normal TIM (for encoder) 18
    f. Normal TIM (for encoder base timer) 20
    g. Normal TIM (for PWM) 21
    h. UART (same for USART) 22

1) Architecture

The goal with this code is to make it the simplest and most efficient possible to be used in various conditions by higher level firmware.

To do this, I split the fonctions in files and the motors and encoders are structures that behave similarly to C++ classes..

a) Diagrams



Motor

Values :

- Type

Normal timer	High Res Timer
- PWMA timer + channel - PWMB timer + channel	- Specific timer (TIMA, TIMB,…)

Functions :

- Motor_Init_normal (TIM_HandleTypeDef *pwm_a_timer, TIM_HandleTypeDef *pwm_b_timer, uint32_t pwm_a_channel, uint32_t pwm_b_channel)
  Initializes a normal timer with every channel free to choose
- Motor_Init_HR (uint32_t hr_timer, uint32_t output_mask)
  Initializes a high-resolution timer, needs the adress to the timer (HRTIM_TIMERINDEX_TIMER_C) and a 4 bytes values that is a mask to activate the correct outputs (HRTIM_OUTPUT_TC1 | HRTIM_OUTPUT_TC2)
- Set_speed (float pwmDuty)
  Same function for both types of timer. pwmDuty is between 0 and the max period value (normally 7999)

Encoder

Values :

- Encoder timer
- Number of lines
- Resolution
- Multiplier (rotor to shaft)
- Speed
- Position
- Alpha (coefficient for low pass filter)

Functions :

- Encoder_Init(TIM_HandleTypeDef *ecod_timer, uint32_t ecod_lines, uint32_t multipl)
  Initializes an Encoder which is defined differently for moon or Faulhaber motors (number of lines and reductor change)
- HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
  Called by the base timer (TIM6) to start calculations with precise time intervals
- Encoder_Reset(Encoder_t *self);
  Currently unused, needed to reset position once we will have implemented automatic position reset

ADC

Values :

All defined manually in analog.c
Not a very modular approach, but increases significantly efficiency and simplifies code

Functions :

- Analog_Init()
  Initializes all opamps and ADCs. Has to be changed manually for each ADC set up
- HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc))
  Called when the conversion is done on an adc, raises a flag (adcx_ready)
- ADC_Store()
  Called in main, polls the values, if all flags are raised, updates the new current and battery level values

b) Notes

- The analog file is impossible to use as a plug in I don't use without adding insane complexity because every value in each buffer has to be multiplied by a specific value then stored to the right place. As a result, it functions for this specific set up but is often changed.
- There are a lot of 'extern ' declarations in each .h file. They are needed to have the compiler recognize the objects generated by the HAL library.
- The safety check is often skipped in testing because the drivers already have a hardware safety for current
- In C, all functions need to have the structure passed as an argument (hence the Encoder_t *self for example)

2) Functions and pins

a) Analog

Board PIN	MCU PIN	Function
Curr1	PA3	OPAMP1 (ADC1)
Curr2	PA7	OPAMP2 (ADC2)
Curr3	PB0	OPAMP3 (ADC2)
Curr4	PD11	OPAMP4 (ADC5)
Curr5	PE13	ADC3_IN3
Curr6	PE14	ADC4_IN1
Curr7	PC3	OPAMP5 (ADC5)
Curr8	PD9	OPAMP6 (ADC4)
Batt_lvl	PE15	ADC4_IN2

b) Encoders

Board PIN	MCU PIN	Function
Ecod1_A	PA15	TIM8_CH1
Ecod1_B	PB8	TIM8_CH2
Ecod2_A	PD3	TIM2_CH1
Ecod2_B	PD4	TIM2_CH2
Ecod3_A	PB6	TIM4_CH1
Ecod3_B	PB7	TIM4_CH2
Ecod4_A	PE2	TIM20_CH1
Ecod4_B	PE3	TIM20_CH2
Ecod5_A	PC0	TIM1_CH1
Ecod5_B	PC1	TIM1_CH2
Ecod6_A	PB5	LPTIM1_IN1
Ecod6_B	PC2	LPTIM1_IN2
Ecod7_A	PA0	TIM5_CH1
Ecod7_B	PA1	TIM5_CH2
Ecod8_A	PA6	TIM3_CH1
Ecod8_B	PA4	TIM3_CH2

c) PWM

Board PIN	MCU PIN	Function
PWM1_A	PA8	HRTIM_CHA1
PWM1_B	PA9	HRTIM_CHA2
PWM2_A	PA11	HRTIM_CHB2
PWM2_B	PA10	HRTIM_CHB1
PWM3_A	PC6	HRTIM_CHF2
PWM3_B	PC7	HRTIM_CHF1
PWM4_A	PC9	HRTIM_CHE2
PWM4_B	PC8	HRTIM_CHE1
PWM5_A	PB12	HRTIM_CHC1
PWM5_B	PB13	HRTIM_CHC2
PWM6_A	PB15	HRTIM_CHD2
PWM6_B	PB14	HRTIM_CHD1
PWM7_A	PF9	TIM15_CH1
PWM7_B	PF10	TIM15_CH2
PWM8_A	PE0	TIM16_CH1
PWM8_B	PE1	TIM17_CH1

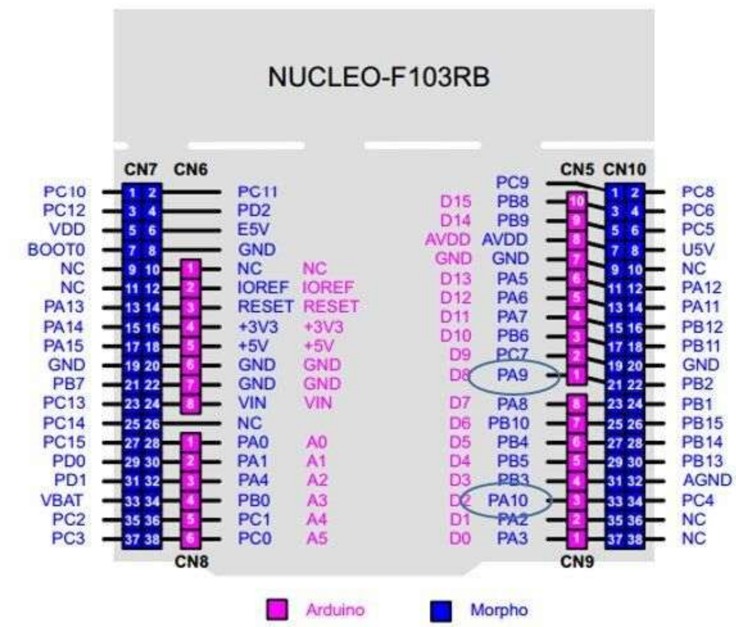
d) Communication

Pin on board	Pin on MCU	Function
UART_EMG_RX	PC11	UART4_RX
UART_EMG_TX	PC10	UART4_TX
UART_Hapt_RX	PD6	USART2_RX
UART_Hapt_TX	PD5	USART2_TX
SWCLK	PA14	SYS_SWCLK
SWDIO	PA13	SYS_SWDIO

e) Nucleo Pinout

Our nucleo G474 has the same pins as this F103

Figure 13. NUCLEO-F103RB

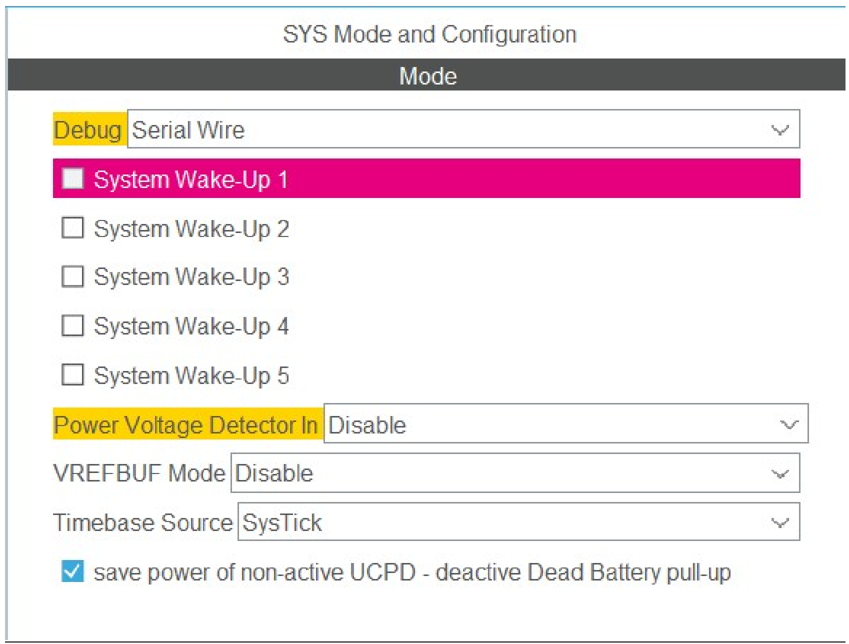


4) CubeMX Set up

Cube MX is very sensitive and horribly documented or understood by AIs. As such, I recommend following precisely these settings to begin, then correct and improve.

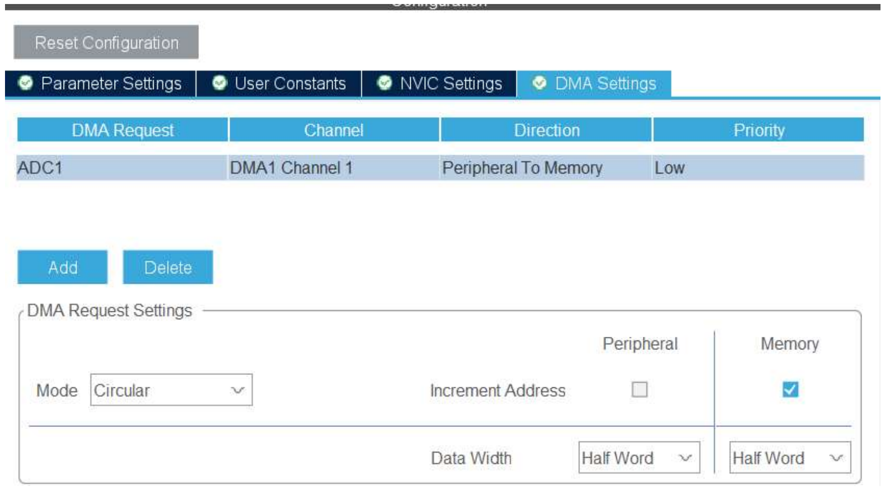
If something is not visible in here, it is left to default. A lot of the settings here are also set to default too.

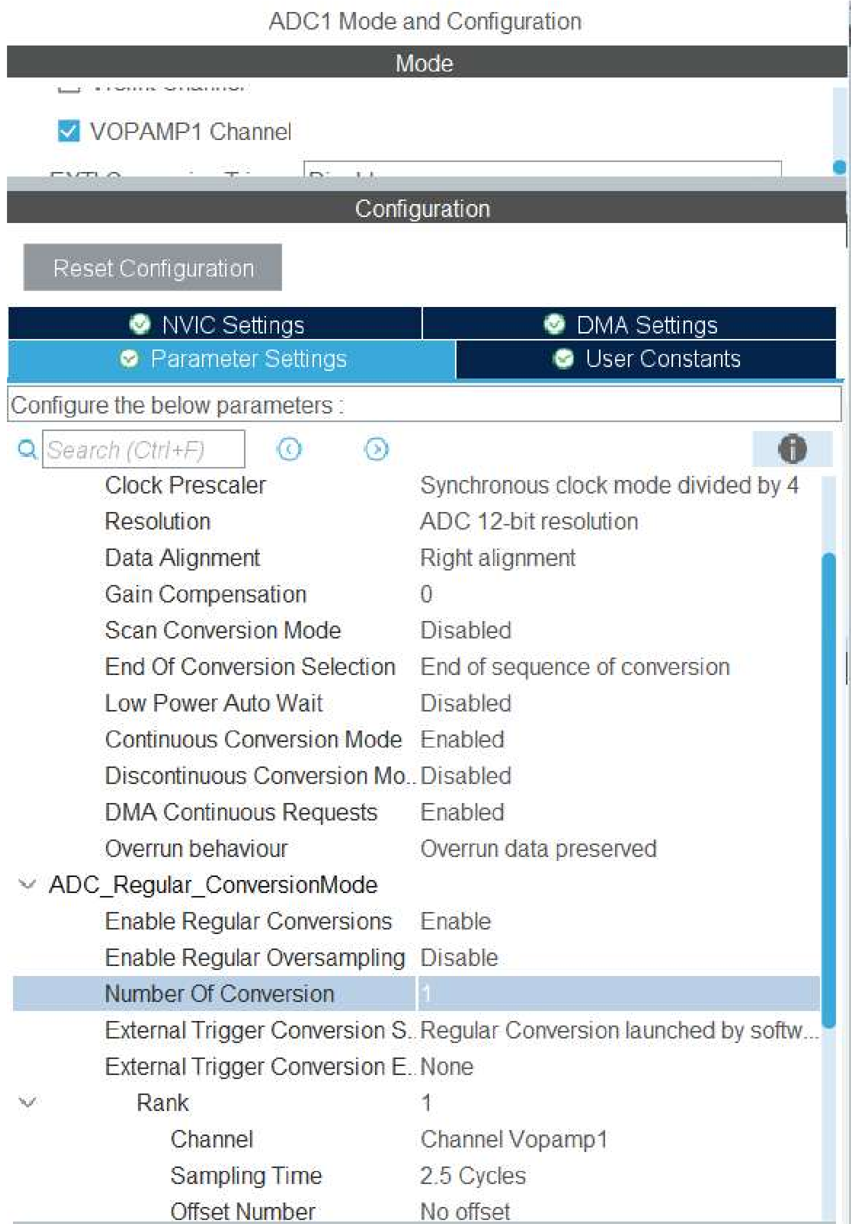
a) Debug



b) ADC (analog converter)

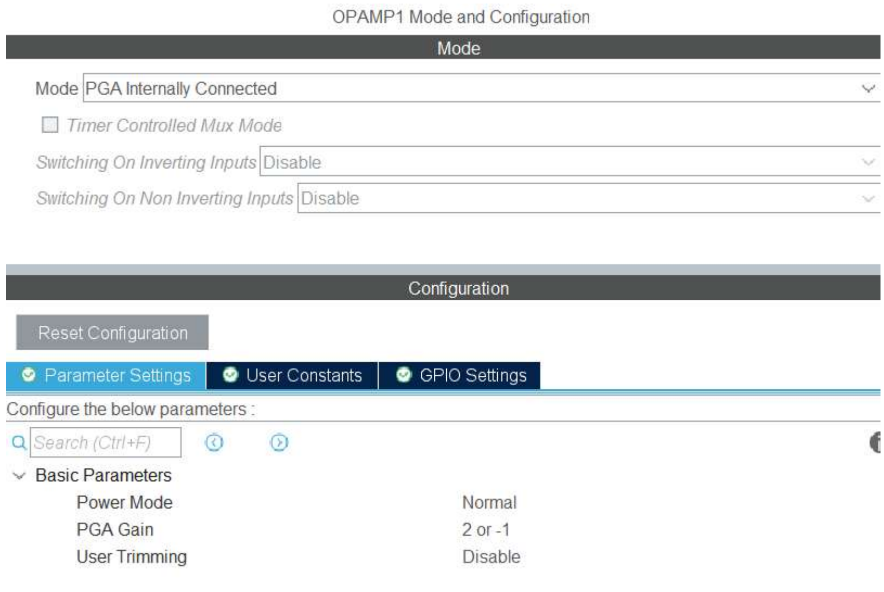
If conversion is garbage, try changing the sampling time to the max (~600 cycles)





ADC1 Mode and Configuration
Mode
VOPAMP1 Channel
Configuration
Reset Configuration
NVIC Settings
DMA Settings
Parameter Settings
User Constants
Configure the below parameters :
Search (Ctrl+F)
Clock Prescaler
Synchronous clock mode divided by 4
Resolution
ADC 12-bit resolution
Data Alignment
Right alignment
Gain Compensation
0
Scan Conversion Mode
Disabled
End Of Conversion Selection
End of sequence of conversion
Low Power Auto Wait
Disabled
Continuous Conversion Mode
Enabled
Discontinuous Conversion Mo..
Disabled
DMA Continuous Requests
Enabled
Overrun behaviour
Overrun data preserved
ADC_Regular_ConversionMode
Enable Regular Conversions
Enable
Enable Regular Oversampling
Disable
Number Of Conversion
1
External Trigger Conversion S..
Regular Conversion launched by softw...
External Trigger Conversion E..
None
Rank
1
Channel
Channel Vopamp1
Sampling Time
2.5 Cycles
Offset Number
No offset

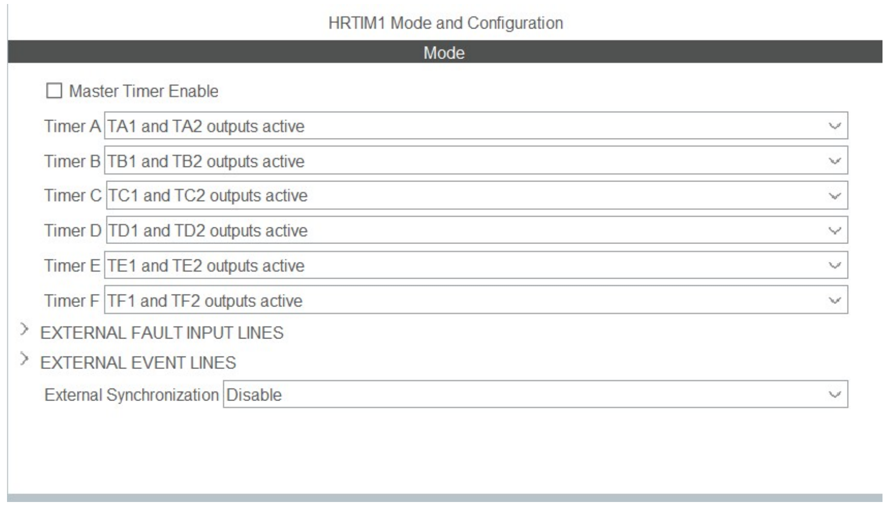
c) OPAMP (signal amplificatory)



We can increase the gain but should not have the signal exceed the 3.3v of the ADC

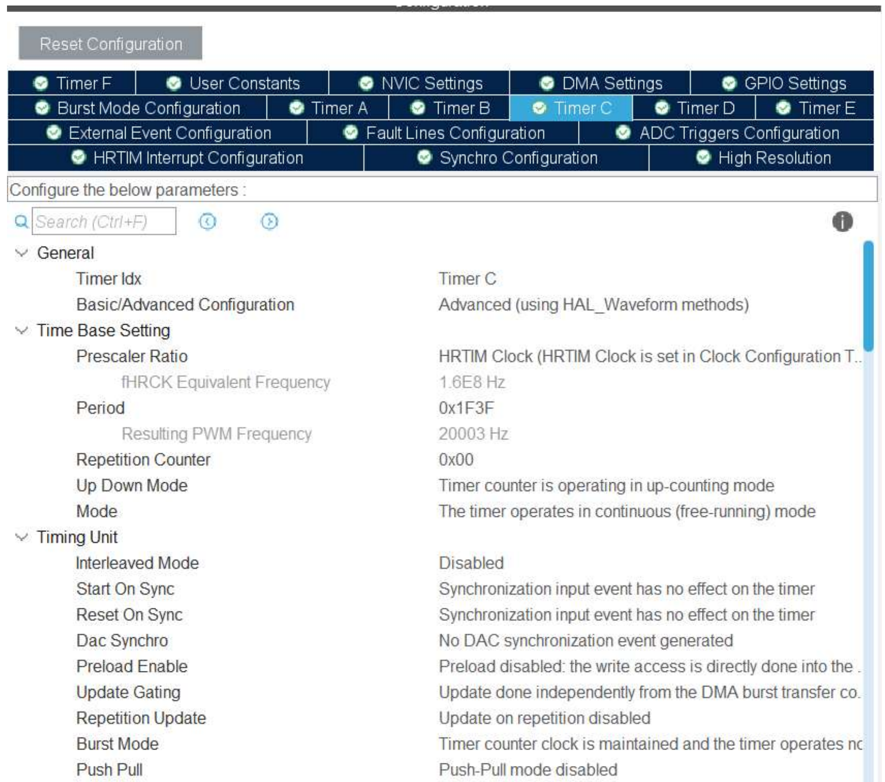
d) High Res Timer (PWM)

general :



For each timer :

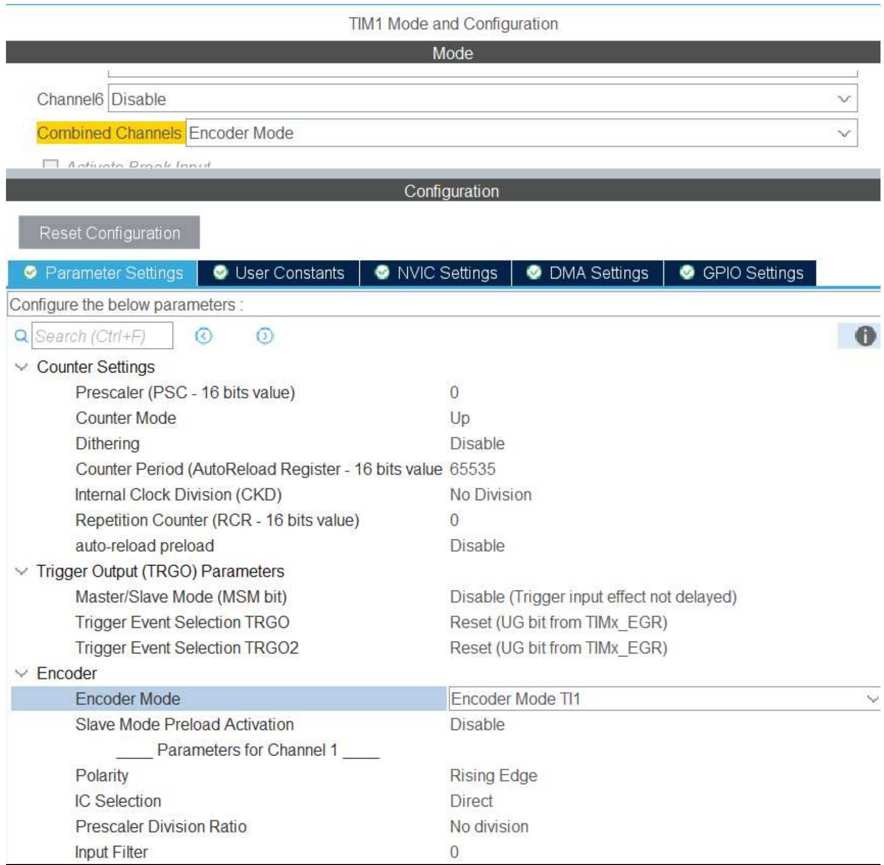
change period depending on clock frequency (160 MHz internal clock on custom, 170MHz on nucleo)


Reset Configuration
Timer F
User Constants
NVIC Settings
DMA Settings
GPIO Settings
Burst Mode Configuration
Timer A
Timer B
Timer C
Timer D
Timer E
External Event Configuration
Fault Lines Configuration
ADC Triggers Configuration
HRTIM Interrupt Configuration
Synchro Configuration
High Resolution
Configure the below parameters :
Search (Ctrl+F)
General
Timer Idx
Timer C
Basic/Advanced Configuration
Advanced (using HAL_Waveform methods)
Time Base Setting
Prescaler Ratio
HRTIM Clock (HRTIM Clock is set in Clock Configuration T...
fHRCK Equivalent Frequency
1.6E8 Hz
Period
0x1F3F
Resulting PWM Frequency
20003 Hz
Repetition Counter
0x00
Up Down Mode
Timer counter is operating in up-counting mode
Mode
The timer operates in continuous (free-running) mode
Timing Unit
Interleaved Mode
Disabled
Start On Sync
Synchronization input event has no effect on the timer
Reset On Sync
Synchronization input event has no effect on the timer
Dac Synchro
No DAC synchronization event generated
Preload Enable
Preload disabled: the write access is directly done into the ...
Update Gating
Update done independently from the DMA burst transfer co...
Repetition Update
Update on repetition disabled
Burst Mode
Timer counter clock is maintained and the timer operates nc
Push Pull
Push-Pull mode disabled

Push Pull	Push-Pull mode disabled
Number of Faults to enable	0
Fault Lock	Timer fault enabling bits are read/write
Dead Time Insertion	Output 1 and output 2 signals are independent
Delayed Protection Mode	No action
Update Trigger Sources Selection : Please enter the	0
Reset Update	Update by Timer reset / roll-over disabled
Resynchronized Update	Update taken into account immediately
Reset Trigger Sources Selection : Please enter the n	0
Interrupt Requests Sources Selection : Please enter t	0
Number of Timer C Internal DMA Request Sources -	0
Compare Unit 1	
Compare Unit 1 Configuration	Enable
Compare Value	0xFFFD
Greater-than comparison	Timer Compare 1 event is generated when counter is equal
Compare Unit 2	
Compare Unit 2 Configuration	Enable
Triggered-Half Mode	Timer Compare 2 register is behaving in standard mode
Compare Value	0xFFFD
Auto Delayed Mode	standard compare mode
Compare Unit 3	
Compare Unit 3 Configuration	Disable

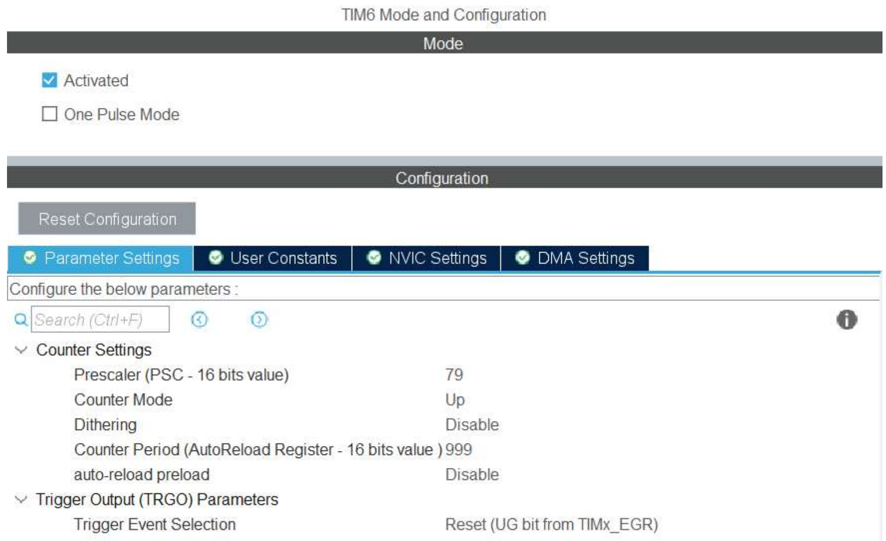
Output 1 Configuration	
Output1 Configuration	TC1
Polarity	Output is active HIGH
Set Source Selection : Please enter the number of A	1
1st Set Source	Timer period event forces the output to its active state
Reset Source Selection : Please enter the number of	1
1st Reset Source	Timer compare 1 event forces the output to its inactive state
Idle Mode	The output is not affected by the burst mode operation
Idle Level	Output at inactive level when in IDLE state
Fault Level	The output is not affected by the fault input
Chopper Mode Enable	Output signal is not altered
Burst Mode Entry Delayed	The programmed Idle state is applied immediately to the Outp
Output 2 Configuration	
Output2 Configuration	TC2
Polarity	Output is active HIGH
Set Source Selection : Please enter the number of A	1
1st Set Source	Timer period event forces the output to its active state
Reset Source Selection : Please enter the number of	1
1st Reset Source	Timer compare 2 event forces the output to its inactive state
Idle Mode	The output is not affected by the burst mode operation
Idle Level	Output at inactive level when in IDLE state
Fault Level	The output is not affected by the fault input

e) Normal TIM (encoder)



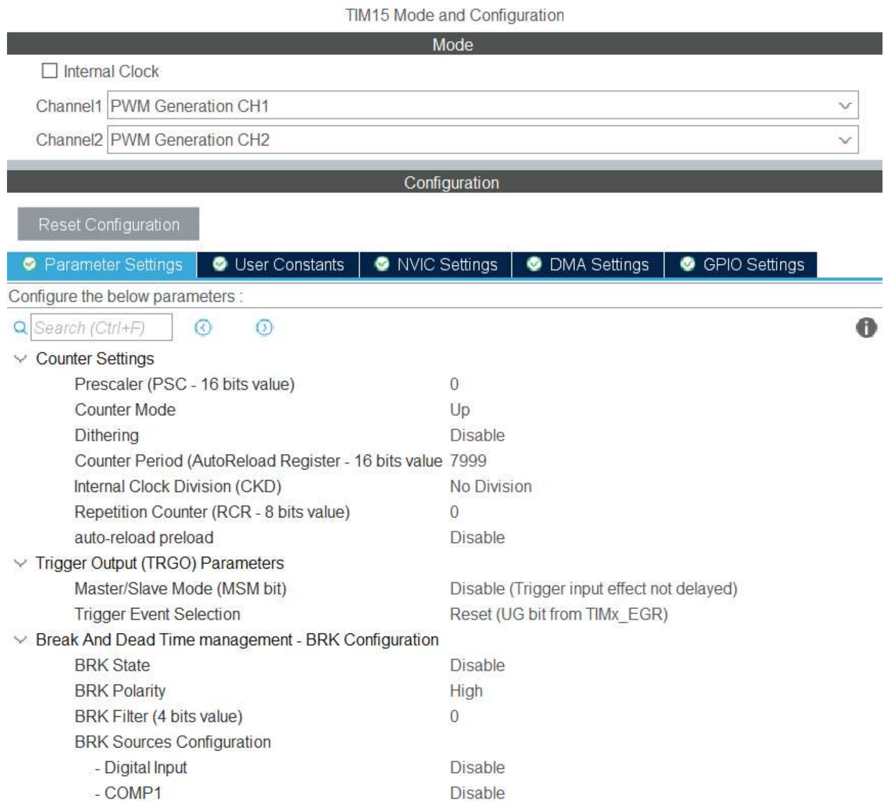
f) Normal TIM (base timer)

Change prescaler depending on clock frequency (79 on custom, 84 on nucleo)



g) Normal TIM (PWM)

Change period depending on clock frequency (7999 on custom, 8499 on nucleo)



h) UART

It is possible to change the Baud rate to match your other device